

~\OneDrive\Desktop\4 year 1 semester\IT4123(P) Agent Based Computing\for\_practice\order\BookServerAgent.java

```
1  import jade.core.*;
2  import jade.core.behaviours.*;
3  import jade.lang.acl.ACLMessage;
4  import jade.domain.FIPAAgentManagement.ServiceDescription;
5  import jade.domain.FIPAAgentManagement.DFAgentDescription;
6  import jade.domain.DFService;
7  import jade.domain.FIPAException;
8  import jade.util.Logger;
9  import java.util.*;
10
11
12  public class BookServerAgent extends Agent {
13
14      private Logger myLogger = Logger.getLogger(getClass().getName());
15      private Map<String,Integer> inventor;
16      private Stack<String> history = new Stack<>();
17      String orderId = UUID.randomUUID().toString();
18
19
20
21      private class WaitPingAndReplyBehaviour extends CyclicBehaviour {
22
23          public WaitPingAndReplyBehaviour() {
24              super();
25          }
26
27          public void action() {
28              ACLMessage msg = myAgent.receive();
29              if(msg != null){
30                  ACLMessage reply = msg.createReply();
31                  String sender = msg.getSender().getLocalName();
32
33                  if(msg.getPerformative()== ACLMessage.REQUEST){
34                      String content = msg.getContent();
35                      if (content.equals("MENU")){
```

```

36         myLogger.log(Logger.INFO, "Agent "+getLocalName()+" - Received PING Request from
"+msg.getSender().getLocalName());
37         reply.setPerformative(ACLMessage.INFORM);
38         reply.setContent(bookList());
39     }
40     else if(content.contains("ORDER|")){
41         myLogger.log(Logger.INFO, "Agent "+getLocalName()+" - Received PING Request from
"+msg.getSender().getLocalName());
42         String result = handleOrderRequest(content,sender);
43         if(result.contains("SUCCESS")){
44             reply.setPerformative(ACLMessage.INFORM);
45         }
46         else{
47             reply.setPerformative(ACLMessage.REFUSE);
48         }
49         reply.setContent(result);
50     }
51     else if(content.equals("HISTORY")){
52         myLogger.log(Logger.INFO, "Agent "+getLocalName()+" - Received PING Request from
"+msg.getSender().getLocalName());
53         reply.setPerformative(ACLMessage.INFORM);
54         reply.setContent(bookHistory());
55     }
56     else{
57         myLogger.log(Logger.INFO, "Agent "+getLocalName()+" - Unexpected request [" +content+"] received from
"+msg.getSender().getLocalName());
58         reply.setPerformative(ACLMessage.REFUSE);
59         reply.setContent("( UnexpectedContent (" +content+"))");
60     }
61
62 }
63 else {
64     myLogger.log(Logger.INFO, "Agent "+getLocalName()+" - Unexpected message
["+ACLMessage.getPerformative(msg.getPerformative())+"] received from "+msg.getSender().getLocalName());
65     reply.setPerformative(ACLMessage.NOT_UNDERSTOOD);
66     reply.setContent("( (Unexpected-act "+ACLMessage.getPerformative(msg.getPerformative())+") )");
67 }
68 send(reply);

```

```
69         }
70         else {
71             block();
72         }
73     }
74 } // END of inner class WaitPingAndReplyBehaviour
75
76 private String bookList(){
77     StringBuilder sb = new StringBuilder("Menu||");
78     for(Map.Entry<String,Integer> entry:inventor.entrySet()){
79         String bookName = entry.getKey();
80         Integer quantity = entry.getValue();
81
82         sb.append(bookName).append(":").append(quantity).append("||");
83     }
84
85     return sb.toString();
86
87 }
88
89 private String handleOrderRequest(String content,String clientName){
90     String[] parts = content.split("\\|");
91     String name = parts[1];
92     Integer quantity ;
93     try{
94         quantity = Integer.parseInt(parts[2]);
95     } catch(NumberFormatException e){
96         return "Invalid Quntity Format";
97     }
98
99     if(!inventor.containsKey(name)){
100         return "Not Found";
101     }
102     if(inventor.get(name) < quantity){
103         return "Insufficient Stock";
104     }
105
106     int newStock = inventor.get(name) - quantity;
```

```

107     inventor.put(name,newStock);
108     String record = clientName + ":" + name + ":" + quantity;
109     history.push(record);
110
111     return "SUCCESS|" + clientName + "|ORDER_ID|" + orderId;
112
113
114 }
115
116 public String bookHistory(){
117     StringBuilder sb = new StringBuilder("History||");
118     Stack<String> temp = new Stack<>();
119
120     while(!history.isEmpty()){
121         String record = history.pop();
122         sb.append(record).append("||");
123         temp.push(record);
124     }
125
126     while(!temp.isEmpty()){
127         history.push(temp.pop());
128     }
129
130     return sb.toString();
131 }
132
133
134 protected void setup() {
135     inventor = new HashMap<>();
136
137     inventor.put("concertA",12);
138     inventor.put("concertB",10);
139     inventor.put("concertC",5);
140     inventor.put("concertD",15);
141     // Registration with the DF
142     DFAgentDescription dfd = new DFAgentDescription();
143     ServiceDescription sd = new ServiceDescription();
144     sd.setType("book-selling");

```

```
145     sd.setName("BookServer");
146     dfd.setName(getAID());
147     dfd.addServices(sd);
148     try {
149         DFService.register(this,dfd);
150         addBehaviour(new WaitPingAndReplyBehaviour());
151     } catch (FIPAException e) {
152         myLogger.log(Logger.SEVERE, "Agent "+getLocalName()+" - Cannot register with DF", e);
153         doDelete();
154     }
155 }
156 }
157
```